AI RECEPTION PRO • V18

Dossier complet — Analyse, conception et développement

Recueil des besoins • Merise / UML • Architecture • Réalisation de A à Z

Édition documentaire : octobre 2026 | Base : archive source fournie | Document de référence, à valider avant diffusion.

Périmètre : deux interfaces (administration et client), application web Angular/Ionic, applications Android Capacitor et services Python/SQLite.

1. Synthèse exécutive

AI Reception Pro V18 est une solution de réception numérique combinant gestion administrative, parcours client, rendez-vous, communications et assistance conversationnelle. Le présent dossier constitue une reconstruction technique à partir des sources livrées, et non un historique attesté des décisions de développement.

2. Méthode et traçabilité

Analyse des fichiers package.json, angular.json, capacitor.config.ts, src/app/app.component.ts, src/app/api.ts, src/app/store.ts, src/app/database.service.ts, src/app/local-auth.service.ts, local_server_development.py et APK_VARIANTS. Les anciens README décrivent des états antérieurs et ne prévalent pas sur le code actuel.

3. Recueil des besoins — démarche structurée

Les besoins ci-dessous sont déduits des modules présents. Ils doivent être confirmés par les parties prenantes lors d'ateliers de cadrage.

Acteur	Objectifs et irritants à explorer
Responsable / administrateur	Centraliser clientèle, planning, services, décisions et échanges ; éviter doubles saisies et rendez-vous conflictuels
Client final	Demander un créneau, suivre le statut, contacter un professionnel sans voir d'autres comptes
Exploitant technique	Installer, sécuriser, sauvegarder, surveiller et mettre à jour la plateforme
Responsable conformité	Limiter la collecte, informer, gérer droits et durées de conservation

3.1. Techniques de collecte

- Entretiens semi-directifs avec responsables et clients : fréquence des rendez-vous, irritants, niveaux d'autonomie.
- Observation des processus existants : réservation, annulation, réponse à un client, clôture de journée.
- Ateliers de priorisation MoSCoW : indispensable, souhaitable, optionnel, hors périmètre.
- Prototypes de parcours Admin et Client, scénarios de test et validation des écrans.
- Matrice de traçabilité : besoin → règle métier → module → test → résultat.

3.2. Exigences fonctionnelles

ID	Exigence	Critère de recette
F01	Authentifier les rôles Admin/Client	Refuser l’accès aux fonctions non autorisées
F02	Gérer les clients et prestations	CRUD et validations cohérentes
F03	Gérer les rendez-vous	Détection des conflits et suivi de statut
F04	Recevoir les demandes Client	Demande visible au bon administrateur
F05	Échanger messages et appels	Destinataires limités au périmètre autorisé
F06	Consulter historique et assistant	Historique lié au bon compte
F07	Fonctionner sur Android	Paquets distincts Admin et Client

3.3. Exigences non fonctionnelles

- Sécurité : autorisation côté serveur, séparation stricte par compte, gestion des sessions et secrets.
- Disponibilité : comportement contrôlé lors des pertes de réseau et erreurs de synchronisation.
- Qualité : messages d’erreur lisibles, navigation responsive, FR/EN selon les modules.
- Maintenance : dépendances verrouillées, journal de versions, sauvegardes et procédure de reprise.
- Protection des données : minimisation, information et politique de conservation à définir.

4. Analyse métier et règles de gestion

Un client peut déposer une demande ; le statut de demande doit être distingué d’un rendez-vous confirmé. L’administrateur gère le catalogue, les disponibilités et les décisions. Toute action sur une ressource doit vérifier le propriétaire ou le périmètre de rattachement côté API.

5. UML — cas d’utilisation

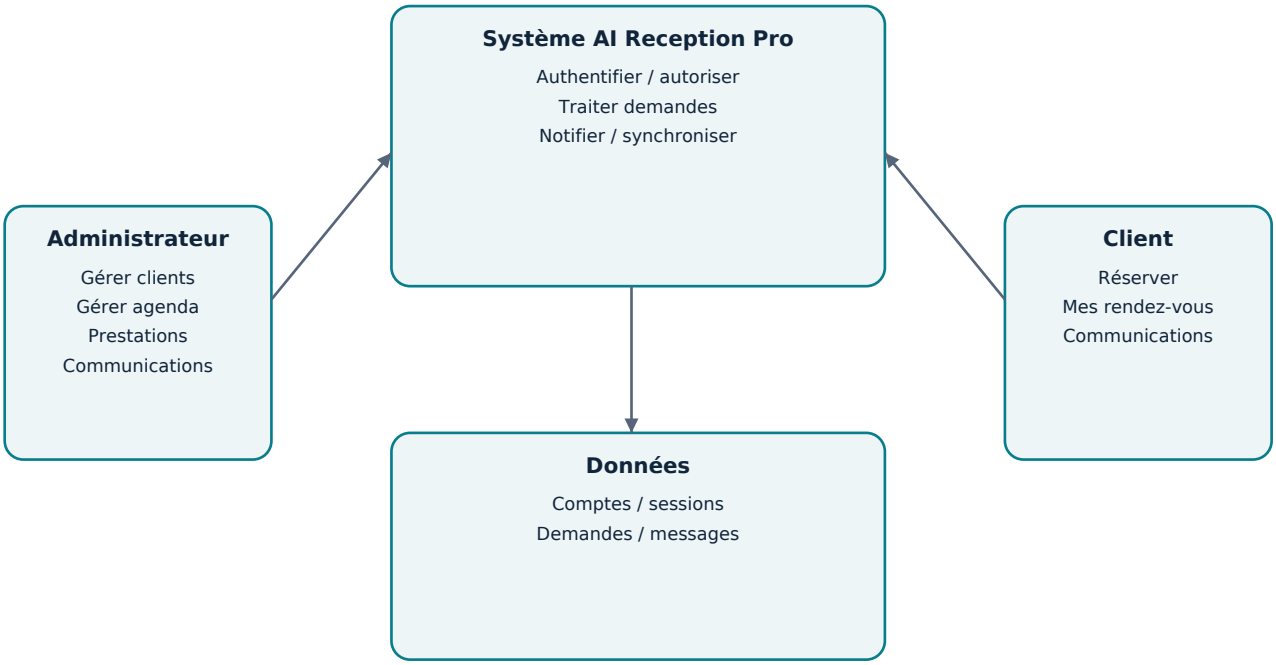
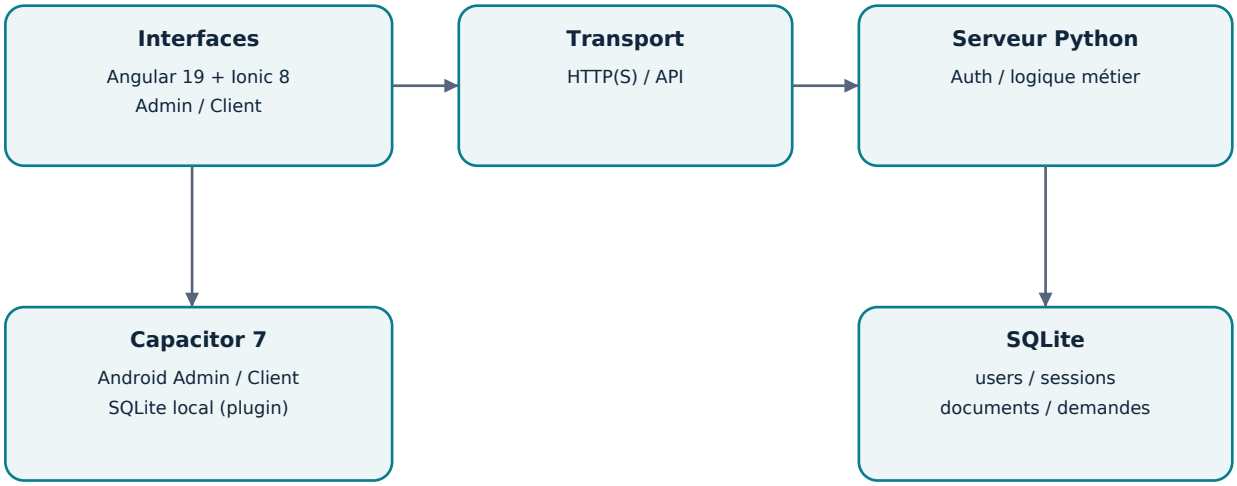


Diagramme conceptuel de cas d’utilisation : les rôles représentés décrivent le périmètre cible, pas une preuve de disponibilité de chaque action dans toutes les installations.

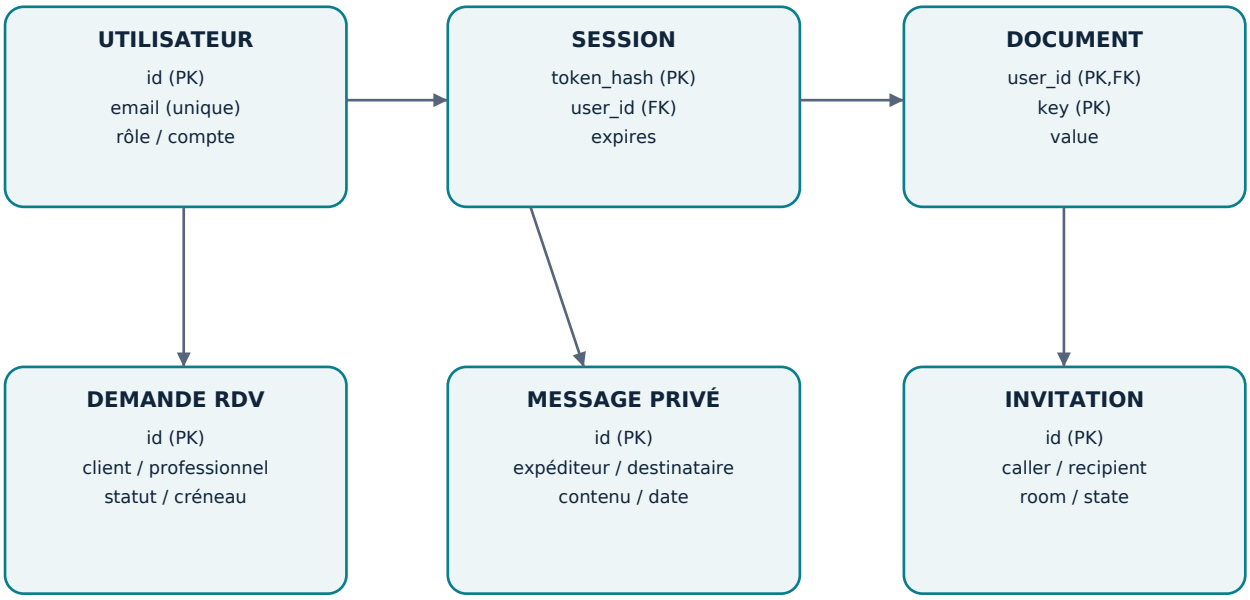
6. Architecture logique et déploiement



Le frontend Angular/Ionic peut fonctionner dans le navigateur ou dans les conteneurs Android Capacitor. Les API Python traitent l’authentification, les opérations métier et les échanges persistés en SQLite. Les services externes de voix/IA ou de communication nécessitent une configuration et des vérifications spécifiques.

7. Modélisation Merise — MCD / MLD simplifié

Le diagramme suivant illustre les entités observées dans le backend. Les cardinalités et certaines associations doivent être validées contre le schéma SQL intégral et les contraintes métier avant d'en faire un MCD normatif.



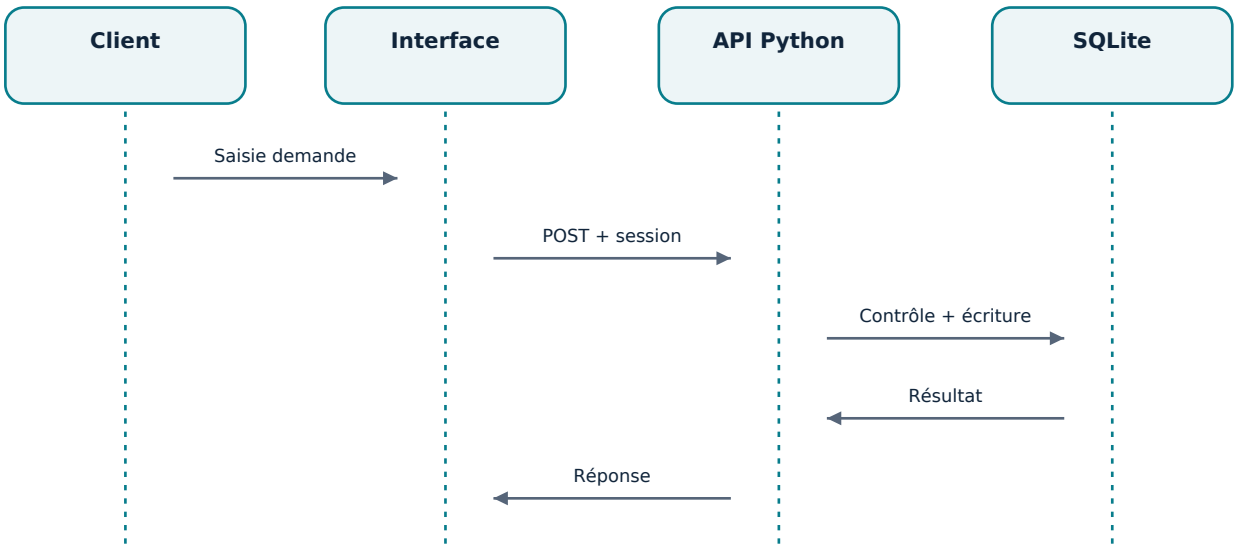
7.1. Dictionnaire de données — entités attestées

Table	Colonnes ou fonction observées
users	id, email, salt, password, created
sessions	token_hash, user_id, expires
documents	user_id, key, value ; clé primaire composite
client_booking_requests	Demandes de rendez-vous clients
private_messages	Messagerie privée
invitations	Invitations de communication
communication_history	Historique des communications
professional_emails	Messagerie professionnelle
notification_reads / notification_deleted	État de consultation / suppression des notifications

7.2. Règles d’intégrité

- La clé (user_id, key) de documents empêche les doublons par utilisateur et clé.
- Le lien session → utilisateur permet d’associer les opérations à une identité authentifiée.
- Les statuts métier et l’unicité des créneaux doivent être vérifiés dans les transactions et contrôles API.
- Les tables supplémentaires de réservation, messages et notifications requièrent un inventaire de colonnes complet pour un MLD définitif.

8. UML — séquence de réservation



Séquence illustrative : l’API doit vérifier la session, le rôle, la disponibilité et les droits avant persistance, puis renvoyer un état métier explicite.

9. Conception applicative

Couche	Fichiers repères	Responsabilité
Interface	src/app/app.component.ts	Navigation et interactions des deux parcours
Accès API	src/app/api.ts	Appels HTTP et échanges avec le backend
État applicatif	src/app/store.ts	Gestion des données et synchronisation côté interface
Persistance locale	src/app/database.service.ts	Gestion locale via plugin SQLite selon plateforme
Authentification locale	src/app/local-auth.service.ts	Comportements d’accès locaux selon configuration
Backend	local_server_development.py	Routes, règles, authentification et SQLite
Packaging	APK_VARIANTS/android-*	Applications Android séparées

10. Développement de A à Z — parcours reproductible

10.1. Phase 0 — cadrage

Définir acteurs, périmètre, parcours prioritaires, contraintes de sécurité, budget et critères d’acceptation.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.2. Phase 1 — spécifications

Écrire user stories, cas nominaux et exceptions, règles de réservation, rôles et dictionnaire des données.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.3. Phase 2 — maquettes

Créer les écrans d’accueil, authentification, tableau de bord, agenda, réservation et messagerie ; valider les parcours.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.4. Phase 3 — socle Angular

Initialiser Angular 19 et Ionic 8 ; définir routes, composants, styles, langue et gestion d’erreurs.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.5. Phase 4 — modèle de données

Créer SQLite, tables utilisateurs/sessions/documents et entités métier ; écrire migrations et tests d’intégrité.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.6. Phase 5 — API Python

Implémenter endpoints, authentification, autorisations par rôle et compte, validation des champs et journalisation.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.7. Phase 6 — interface Admin

Développer clients, prestations, agenda, demandes, paramètres, communications et historiques.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.8. Phase 7 — interface Client

Développer réservation publique, authentification client, suivi des demandes et communications autorisées.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.9. Phase 8 — synchronisation

Gérer erreurs HTTP, état local, conflits, rafraîchissement, sessions expirées et reprise après déconnexion.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.10. Phase 9 — assistant et communications

Intégrer les services réellement configurés, permissions micro/caméra, gestion des échecs et historiques.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.11. Phase 10 — Android

Configurer Capacitor, séparer applicationId, assets, manifestes et construire les deux variantes.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.12. Phase 11 — qualité et sécurité

Tester rôles, accès intercomptes, CRUD, calendrier, confidentialité, dépendances et restauration.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

10.13. Phase 12 — publication et transfert

Produire AAB signés, fiches de diffusion, documentation, inventaire et dossier de recette.

- Livrable : fiche de validation et résultat vérifiable pour cette phase.
- Point de contrôle : aucune donnée réelle ni secret dans les jeux d’essai ou les dépôts publics.

11. Sécurité et confidentialité

- Séparer les données par identité et rôle côté serveur, sans se fier uniquement à l’interface.
- Contrôler les paramètres de session, expiration, réinitialisation et stockage des secrets.
- Protéger les sauvegardes SQLite et définir un plan de reprise.
- Contrôler les autorisations Android (microphone, caméra, biométrie) et les politiques de sauvegarde.
- Auditer les fichiers historiques, archives imbriquées et configurations avant diffusion.

12. Tests de recette détaillés

Cas	Procédure	Attendu
T01	Connexion Admin puis accès à un module de gestion	Autorisé
T02	Connexion Client puis tentative d’accès à une API Admin	Refus côté serveur
T03	Client A demande des données du client B	Accès refusé
T04	Demande sur créneau déjà réservé	Conflit géré
T05	Annulation d’une demande	État et notifications cohérents
T06	Coupure réseau pendant modification	Erreur visible et reprise contrôlée
T07	Redémarrage serveur et sauvegarde/restauration	Données cohérentes
T08	Appel sans permission microphone	Message explicite, pas de blocage

13. Dossier de livraison commerciale

- Code source propre, sans secrets ni sauvegardes anciennes.
- Manuels d’utilisation et d’installation FR/EN, dossier d’architecture et de conception.
- Liste des dépendances et vérification de leurs licences.
- Journal des versions, liste des fonctionnalités démontrées, limites connues.
- Procès-verbal de recette et clause de périmètre commercial.

14. Réserves et preuves

Ce document ne prétend pas retracer l’historique réel du recueil des besoins : les étapes sont une démarche de réalisation documentée et reproductible. Les diagrammes sont des modèles de compréhension établis à partir des sources et à valider. Les fonctionnalités dépendantes de services distants, l’état du serveur en production, la conformité réglementaire et les tests sur appareils ne sont pas certifiés par l’analyse statique.